INTERVIEW PREP • 2026 EDITION

Java 21 Advanced Features

Complete Interview Guide

A focused, no-fluff guide to the Java 21 features senior engineers ask about in product-based company interviews. Includes code examples, interview answers, common follow-up questions, and a quick-reference cheat sheet.

What's Inside

- **Virtual Threads** — Scale to millions without thread pool tuning
- **Structured Concurrency** — Fan-out API calls safely
- **Scoped Values** — The modern ThreadLocal replacement
- **Record Patterns** — Nested destructuring
- **Pattern Matching for Switch** — Exhaustive, null-safe branching
- **Sequenced Collections** — Unified first/last/reversed APIs
- **String Templates** — Safe interpolation, SQL-injection-proof
- **Interview Q&A** — Common follow-up questions with answers
- **Cheat Sheet** — One-page quick reference

WHO THIS GUIDE IS FOR

Backend Java developers with 3–8 years of experience preparing for product-based companies (Google, Amazon, Meta, Flipkart, Swiggy, Razorpay)

By @techtribers | Built for Indian developers levelling up

1. Virtual Threads

Lightweight threads managed by the JVM — not the OS

Virtual threads decouple Java threads from OS threads. You can create millions of them without blowing up memory or CPU. They block cheaply, so classic imperative blocking code suddenly scales like reactive code.

Code Example

VirtualThreads.java

```
// OLD: Platform thread costs ~1 MB stack
new Thread(() -> handleRequest()).start();

// NEW: Virtual thread — few KB, millions possible
Thread.startVirtualThread(() -> handleRequest());

// With executor for structured work
try (var exec = Executors
      .newVirtualThreadPerTaskExecutor()) {
  IntStream.range(0, 1_000_000)
    .forEach(i -> exec.submit(() -> task(i)));
}
```

Key Points

- Managed by the JVM, parked on OS threads called 'carrier threads'
- Cheap to create and cheap to block — no need for thread pool tuning
- Best for I/O-bound workloads: REST APIs, database calls, HTTP clients
- NOT a magic bullet for CPU-bound tasks — use platform threads there
- Built into Thread class — no library changes needed

BEST FOR

High-throughput I/O workloads. Replaces reactive programming for most microservice use cases. Your Spring controller method stays synchronous but now scales to millions of concurrent requests.

INTERVIEW ANSWER

"Virtual threads are lightweight JVM-managed threads that block without tying up an OS thread. They make blocking I/O code scale as well as reactive code — without the complexity."

2. Structured Concurrency

Concurrent tasks as a single unit of work

Structured concurrency treats a group of concurrent tasks like a single operation. If the parent is cancelled, all children are cancelled. If one child fails, siblings can be cancelled automatically. No more orphan threads burning CPU after an error.

Code Example

StructuredConcurrency.java

```
try (var scope = new StructuredTaskScope
        .ShutdownOnFailure()) {

  var user  = scope.fork(() -> fetchUser(id));
  var order = scope.fork(() -> fetchOrders(id));
  var cart  = scope.fork(() -> fetchCart(id));

  scope.join();            // wait for all tasks
  scope.throwIfFailed();   // fail fast on error

  return new Response(
    user.get(), order.get(), cart.get()
  );
}
// If ANY task fails, others are auto-cancelled
```

Key Points

- Scope defines a unit of concurrent work — like try-with-resources for threads
- ShutdownOnFailure: first failure cancels all siblings
- ShutdownOnSuccess: first success cancels all siblings (useful for redundant calls)
- Pairs perfectly with virtual threads — each fork uses a VT by default
- Preview feature as of Java 21 — expected to stabilize soon

BEST FOR

Fan-out aggregation calls where you hit multiple backend services and combine responses. If one fails, you don't want orphan requests wasting resources.

INTERVIEW ANSWER

"It's try-with-resources for concurrent tasks. The parent scope owns child tasks, so cancellation, error handling, and lifecycle are predictable — no more leaked threads."

3. Scoped Values

The modern ThreadLocal replacement

Scoped values are immutable context bindings that exist only for a specific scope of execution. They solve ThreadLocal's problems: mutation, memory leaks, and poor performance with virtual threads.

Code Example

ScopedValues.java

```
static final ScopedValue<User> USER
    = ScopedValue.newInstance();

// Bind value for the scope below
ScopedValue.where(USER, currentUser)
    .run(() -> {
       service.process();  // can access USER
    });

// Inside any method within that scope:
if (USER.isBound()) {
    User u = USER.get();   // immutable, fast
}
```

Why ScopedValue > ThreadLocal

- Immutable by design — cannot be mutated after binding
- Auto-cleanup when scope exits — no memory leak risk
- Zero allocation overhead per virtual thread (ThreadLocal has it)
- Inherited correctly by child tasks in structured concurrency
- Thread-safe without synchronization

BEST FOR

Request context propagation — authenticated user, trace ID, tenant info, correlation ID. Critical when using millions of virtual threads where ThreadLocal would leak memory.

INTERVIEW ANSWER

"Scoped values replace ThreadLocal with immutable, scope-bound context. Better performance and safer with virtual threads at scale — ThreadLocal was never designed for millions of threads."

4. Record Patterns

Deconstruct records directly in pattern matching

Record patterns let you destructure a record — extracting its components — in a single expression. Combined with pattern matching, this eliminates chains of getter calls and makes data handling declarative.

Code Example

RecordPatterns.java

```
record Point(int x, int y) {}
record Line(Point start, Point end) {}

// OLD way — getter chains
if (obj instanceof Line line) {
  int x1 = line.start().x();
  int y2 = line.end().y();
  // ... use them
}

// NEW way — nested destructuring
if (obj instanceof
    Line(Point(var x1, var y1),
         Point(var x2, var y2))) {
  drawLine(x1, y1, x2, y2);
}
```

Key Points

- Works inside instanceof AND in switch expressions
- Supports nested patterns — destructure deep structures in one line
- Type-safe: compiler checks record component types
- Use 'var' to let the compiler infer component types
- Only works on record types (for now — future: general destructuring)

BEST FOR

Parsing domain objects, event processing, ADT-style error handling with Result<T>, and any place you'd chain .getX().getY().getZ() today.

INTERVIEW ANSWER

"Record patterns let me destructure nested records directly in instanceof or switch, eliminating boilerplate accessor calls and making code read like declarative data handling."

5. Pattern Matching for Switch

Type patterns, guards, and null handling

Switch is now a powerful pattern matcher. It supports type patterns, guard conditions via 'when', null handling, and exhaustiveness checks with sealed types. This replaces messy visitor patterns and instanceof chains.

Code Example

SwitchPatterns.java

```
sealed interface Shape
       permits Circle, Square, Triangle {}

double area = switch (shape) {
  case Circle c       -> Math.PI * c.r() * c.r();
  case Square s       -> s.side() * s.side();
  case Triangle t
         when t.isRight()
                      -> 0.5 * t.base() * t.h();
  case Triangle t     -> heronFormula(t);
  case null           -> 0.0;  // explicit null!
};
```

Key Points

- Type patterns: 'case Circle c' — matches type AND binds variable
- Guard clauses with 'when' keyword for additional conditions
- Exhaustiveness: compiler verifies all sealed type subtypes are handled
- Explicit null case — no more surprise NullPointerException in switch
- Works as expression (returns value) or statement (side effects)

BEST FOR

Domain modeling with sealed hierarchies, event handlers, state machines, finite-state parsers, and any branching logic that used to need visitor pattern or instanceof chains.

INTERVIEW ANSWER

"Pattern matching in switch gives exhaustiveness with sealed types, guard clauses via 'when', and null handling — making domain logic type-safe and readable."

6. Sequenced Collections

Unified first/last/reversed across ordered collections

New interfaces — SequencedCollection, SequencedSet, SequencedMap — unify how you access first, last, and reversed elements. Ends the API inconsistency that has existed in Java since 1.2.

SequencedCollections.java

```
// OLD: different API for every collection
list.get(0);                  // List first
deque.peekFirst();            // Deque first
linkedSet.iterator().next();  // Set first

// NEW: unified across all ordered collections
list.getFirst();     list.getLast();
list.addFirst(x);    list.addLast(x);
list.reversed();     // reversed VIEW, not copy

// SequencedMap too
map.firstEntry();   map.lastEntry();
map.reversed();
```

Key Points

- Implemented by List, Deque, LinkedHashSet, LinkedHashMap, SortedSet, SortedMap
- reversed() returns a view — cheap, doesn't copy
- Makes generic utility methods much cleaner

7. String Templates

Safe interpolation, SQL-injection-proof (preview)

StringTemplates.java

```
// OLD: error-prone concatenation
String msg = "User " + name
    + " has " + count + " orders";

// NEW: safe interpolation
String msg = STR."User \{name} has \{count} orders";

// SQL — prevents injection automatically
Query q = SQL."""
    SELECT * FROM users
    WHERE email = \{userInput}
""";
```

Key Points

- Processors (STR, SQL, FMT, JSON) handle interpolation safely
- SQL processor binds parameters — no injection risk
- Preview feature — syntax may still evolve slightly

Top Interview Questions & Answers

Q: What's the difference between a virtual thread and a platform thread?

A: Platform threads are thin wrappers around OS threads — expensive (~1 MB stack), limited in number. Virtual threads are managed by the JVM, park onto carrier platform threads when blocking, and cost a few KB. You can have millions of virtual threads; you can't have millions of platform threads.

Q: Are virtual threads faster than platform threads?

A: Per-task, no — the actual work takes the same time. What virtual threads give you is much higher concurrency without tuning thread pools, because blocking a virtual thread doesn't block an OS thread. For I/O-bound workloads this translates to dramatically better throughput.

Q: When should you NOT use virtual threads?

A: CPU-bound workloads — you still have only N cores. Also avoid them when using synchronized blocks on code that holds the lock across long operations, because that pins the virtual thread to its carrier. Use ReentrantLock instead in those cases.

Q: Why is StructuredTaskScope preferred over ExecutorService for fan-out?

A: ExecutorService has no lifecycle relationship between tasks — if one fails, siblings keep running. StructuredTaskScope enforces a parent-child relationship so failure/cancellation propagates automatically, and scope exit guarantees cleanup.

Q: Why can't ThreadLocal scale with virtual threads?

A: Each ThreadLocal entry allocates memory per thread. With millions of virtual threads, you'd allocate millions of entries even if only a few use the value. ScopedValue binds values to a scope instead, so lookup is cheap and cleanup is automatic.

More Questions & Migration Tips

Q: Can I mix record patterns with switch in the same code?

A: Yes — record patterns work in both instanceof and switch. In fact, the most powerful pattern is switch with nested record patterns and guard clauses, giving you exhaustive, type-safe branching that would otherwise take 50+ lines of instanceof plus getters.

Q: What makes a sealed type 'exhaustive' in a switch?

A: A sealed interface or class lists its permitted subtypes at compile time. The switch must cover all of them — if you add a new permitted subtype and forget to handle it, the compiler fails. This catches bugs that would otherwise slip into production.

Q: How do I migrate an existing Spring Boot app to Java 21?

A: Upgrade JDK first, run your test suite. Most code just works. Enable virtual threads for the web server via spring.threads.virtual.enabled=true. Replace ThreadLocal use with ScopedValue where you own the code. Refactor instanceof chains to pattern-matching switches gradually.

Q: What's the performance cost of pattern matching in switch?

A: Essentially zero. The compiler generates bytecode equivalent to a chain of instanceof + cast operations. For sealed types, the JIT can often optimize further because the set of possibilities is bounded.

Q: Should I rewrite all reactive code to virtual threads?

A: No, not automatically. Reactive code remains better for backpressure, streaming, and fine-grained async composition. But for most CRUD microservices, virtual threads give you similar scalability with simpler code. Profile before deciding.

Quick Reference Cheat Sheet

When to Use What

- **Virtual Threads:** I/O-bound workloads with high concurrency
- **Structured Concurrency:** Fan-out API calls with linked lifecycles
- **Scoped Values:** Request context in virtual thread environments
- **Record Patterns:** Destructuring domain objects in branching logic
- **Pattern Matching Switch:** Exhaustive handling of sealed type hierarchies
- **Sequenced Collections:** Generic code that works across List/Deque/LinkedHash
- **String Templates:** Safe SQL queries, logs, formatted output

Common Anti-Patterns to Avoid

- ✗ Using virtual threads for CPU-bound tasks
- ✗ Synchronized blocks inside virtual threads holding locks during I/O (pins the thread)
- ✗ Continuing to use ThreadLocal in virtual-thread-heavy code
- ✗ Generic Exception catches in pattern-matching switches
- ✗ Mixing concatenation and STR templates — pick one style per module

THE ONE-LINER TO MEMORIZE

"Virtual threads + structured concurrency + scoped values are the three features that unlock massive scalability in Java 21."

Follow @techtribers for more interview guides

Daily Java, Spring Boot, Kafka & System Design content

This PDF was delivered automatically via SyncRelay | syncrelay.co